NED UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science & Information Technology

COURSE CODE: CT-159

Course Title:	**Data Structure Algorithms & Applications**	**Assessment:**	Complex Computing Problem (CCP) Visualizer
Instructor:	Furqan Hussain Essani (Reviewed: Dr. M. Mubashir Khan)	**Class & Batch:**	2nd Year (Batch 2025) • Fall 2026 Session

WEEK 01 Introduction, ADT, Linear & Binary Search

`01_arrays_and_searching/`

- [ ] Array ADT — Insert at Index (head `0`, middle `idx`, tail n with right-shift)
- [ ] Array ADT — Delete at Index (with left-shift)
- [ ] Array ADT — Update / Access at Index (`arr[idx] = val`)
- [ ] Array ADT — Reverse In-Place & Rotate Left / Right by k
- [ ] Linear Search (Iterative & Recursive)
- [ ] Binary Search (Iterative & Recursive)
- [ ] Binary Search — First / Last Occurrence (Duplicates / Lower & Upper Bound)

WEEK 02 Singly Linked List

`02_singly_linked_list/`

- [ ] Singly Linked List — Insert at Head, Tail, and Position k
- [ ] Singly Linked List — Delete Head, Tail, and by Value / Position k
- [ ] Singly Linked List — Search by Value & Traverse / Display
- [ ] Singly Linked List — Reverse (Iterative 3-pointer: `prev`, `curr`, `next`) & Recursive
- [ ] Linked List Sorting (Merge Sort / Bubble Sort on node pointer chains)
- [ ] **VIVA STAPLE** Find Middle Node of Linked List (Slow & Fast pointer technique)
- [ ] **VIVA STAPLE** Merge Two Sorted Linked Lists into one sorted list

WEEK 03 Doubly & Circular Linked List

`03_doubly_circular_linked_list/`

- [ ] Doubly Linked List — Insert at Head, Tail, Position k (managing `prev` and `next`)
- [ ] Doubly Linked List — Delete at Head, Tail, and by Value / Position
- [ ] Doubly Linked List — Search, Reverse, and Forward / Backward Bi-directional Traversal
- [ ] Circular Singly Linked List — Insert & Delete at Head and Tail
- [ ] Circular Singly Linked List — Search & Traversal (`do ... while (curr != head)`)
- [ ] **VIVA STAPLE** Floyd's Cycle Detection Algorithm (Tortoise and Hare pointer loop check)

WEEK 04 Recursion & Stack

`04_recursion_and_stack/`

- [ ] Stack (Array-based ADT) — `push`, `pop`, `peek` / `top`, `isEmpty`, `isFull`
- [ ] Stack (Linked-List-based ADT) — Dynamic node allocation at top
- [ ] Recursion Call Stack — Factorial, Fibonacci, Recursive reversal
- [ ] **VIVA STAPLE** Tower of Hanoi (3-peg recursive solution & move visualizer)
- [ ] **VIVA STAPLE** Balanced Parentheses / Bracket Matching (`{[()]}` validation)
- [ ] **VIVA STAPLE** Infix to Postfix Conversion (using operator precedence and stack)
- [ ] **VIVA STAPLE** Postfix Expression Evaluation (operand stack calculation)

WEEK 05 Queue, Circular Queue & Double-Ended Queue

`05_queues_and_deque/`

- [ ] Linear Queue (Array & Linked-List-based) — `enqueue`, `dequeue`, `peek` / `front`
- [ ] Circular Queue (Array-based with `(rear + 1) % capacity`) — Full / Empty boundary logic
- [ ] Double-Ended Queue (Deque) — `insertFront`, `insertRear`, `deleteFront`, `deleteRear`, `getFront`, `getRear`
- [ ] Standard Template Library Comparison — `std::queue<int>` & `std::deque<int>`

WEEK 06 Elementary Sorting Techniques

`06_elementary_sorting/`

- [ ] Bubble Sort — Standard nested loops with pairwise comparison and swap
- [ ] Bubble Sort — Early-exit optimization with `bool swapped` flag (O(n) best-case)
- [ ] Insertion Sort — Shift-based shifting prefix (`arr[j + 1] = arr[j]`) and swap-based variants
- [ ] Selection Sort — Finding `minIdx` per pass and single swap to sorted position

WEEK 07 Advanced Sorting Techniques

`07_advanced_sorting/`

- [ ] Merge Sort — Top-down recursive `mergeSort(arr, l, r)` + auxiliary merge step (O(n log n))
- [ ] Quick Sort — Lomuto Partition scheme (pivot at `arr[high]`)
- [ ] Quick Sort — Hoare Partition scheme & Median-of-Three pivot selection

WEEK 09 Binary Tree

`09_binary_tree/`

- [ ] Binary Tree — Dynamic node creation and Level-Order insertion (via Queue)
- [ ] Tree Traversals — Preorder (NLR), Inorder (LNR), and Postorder (LRN) (Recursive & Iterative)
- [ ] Tree Traversals — Level-Order Traversal (Breadth-First Search using Queue)
- [ ] **VIVA STAPLE** Calculate Tree Height / Maximum Depth (recursive bottom-up)
- [ ] **VIVA STAPLE** Count Total Nodes and Count Leaf Nodes (recursive check `left == null && right == null`)
- [ ] **VIVA STAPLE** Invert / Mirror Binary Tree (swapping left and right child pointers)

WEEK 10 Binary Heap, Heap Sort & Priority Queue

`10_heap_and_priority_queue/`

- [ ] Max-Heap & Min-Heap — Insert operation with `siftUp` / Bubble-Up
- [ ] Max-Heap & Min-Heap — Extract Root with `siftDown` / Bubble-Down
- [ ] Build Heap — O(n) linear-time bottom-up `heapify` from index `n/2 - 1` down to 0
- [ ] Heap Sort — In-place array sorting using Build Max-Heap + root swaps & heapify
- [ ] Priority Queue ADT — Heap-backed `push`, `pop`, and `top`

WEEK 11 Binary Search Tree (BST)

`11_binary_search_tree/`

- [ ] BST — Insert operation (Recursive & Iterative with trailing parent pointer)
- [ ] BST — Search operation & Extremes (`findMin` and `findMax`)
- [ ] BST — Delete Case 1: Deleting a Leaf Node (direct deallocation)
- [ ] BST — Delete Case 2: Node with One Child (bypass linking)
- [ ] BST — Delete Case 3: Node with Two Children (In-Order Successor / Predecessor replacement)
- [ ] BST — Inorder Traversal (producing monotonically sorted output)
- [ ] **VIVA STAPLE** Check if a Binary Tree is a Valid BST (`isValidBST` with min/max range check)
- [ ] **VIVA STAPLE** Lowest Common Ancestor (LCA) in BST

WEEK 12 AVL Tree (Self-Balancing BST)

`12_avl_tree/`

- [ ] AVL Tree — Height & Balance Factor computation (`height(left) - height(right)`)
- [ ] AVL Rotations — LL Imbalance → Right Rotation (`rightRotate`)
- [ ] AVL Rotations — RR Imbalance → Left Rotation (`leftRotate`)
- [ ] AVL Rotations — LR Imbalance → Left-Right Double Rotation (`leftRotate(left)` then `rightRotate(root)`)
- [ ] AVL Rotations — RL Imbalance → Right-Left Double Rotation (`rightRotate(right)` then `leftRotate(root)`)
- [ ] AVL Tree — Delete Node with Recursive Rebalancing Rotations

WEEK 13 Graph — Representation & Traversal

`13_graph_traversal/`

- [ ] Graph Representation — Adjacency Matrix (`int adj[V][V]`) and Adjacency List (`vector<vector<int>>`)
- [ ] Graph Operations — Add / Remove Vertex and Add / Remove Edge (Directed & Undirected)
- [ ] Breadth-First Search (BFS) — Queue-based level exploration with `visited[]` array
- [ ] Depth-First Search (DFS) — Recursive traversal and Iterative stack-based exploration
- [ ] **VIVA STAPLE** Cycle Detection in Undirected Graph (using parent pointer in BFS/DFS)
- [ ] **VIVA STAPLE** Cycle Detection in Directed Graph (using 3-color or recursion stack array)

WEEK 14 Graph — Minimum Spanning Tree & Topological Sorting

`14_graph_mst_toposort/`

- [ ] **PREREQUISITE** Disjoint Set Union (DSU / Union-Find with Path Compression & Union by Rank)
- [ ] Minimum Spanning Tree — Prim's Algorithm (Greedy vertex exploration via `key[]`, `parent[]`, `inMST[]`)
- [ ] Minimum Spanning Tree — Kruskal's Algorithm (Sorted edge list + DSU cycle prevention)
- [ ] Topological Sorting — Kahn's Algorithm (In-Degree counting + Queue BFS on Directed Acyclic Graph)
- [ ] Topological Sorting — DFS Post-Order Stack Method (push to stack on finish)
- [ ] Dijkstra's Shortest Path Algorithm — Single-Source Shortest Path on non-negative weighted graphs

WEEK 15 Hashing — Hash Functions & Collision Resolution

`15_hashing/`

- [ ] Hash Functions — Division Method (`key % m`), Mid-Square Method, Folding Method
- [ ] Open Addressing — Linear Probing (`(hash + i) % m`) with Insert, Search, and Tombstone Deletion
- [ ] Open Addressing — Quadratic Probing (`(hash + i*i) % m`) to reduce primary clustering
- [ ] Open Addressing — Double Hashing (`(h1(k) + i * h2(k)) % m`)
- [ ] Closed Addressing — Separate Chaining (Array of Linked Lists `Node* table[m]`)
- [ ] **VIVA STAPLE** Load Factor Calculation (α = n / m) & Dynamic Rehashing (doubling table to prime size)
